🔄 AI Model Workflow - Visual Guide

Complete Pipeline: Training → Deployment → Inference

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          TRAINING PHASE (One-Time)                       │
└──────────────────────────────────────────────────────────────────────────┘

    Step 1: Run Training Script
    ┌──────────────────────────────┐
    │   python train_model.py      │
    │   (backend/train_model.py)   │
    └──────────────────────────────┘
                   │
                   │
                   ▼
    Step 2: Load Dataset
    ┌──────────────────────────────┐
    │   GoEmotions Dataset         │
    │   • 58,000 Reddit texts      │
    │   • 27 emotions labeled      │
    └──────────────────────────────┘
                   │
                   │
                   ▼
    Step 3: Load Pre-trained Model
    ┌──────────────────────────────┐
    │   distilbert-base-uncased    │
    │   (from HuggingFace)         │
    │   • 66M parameters           │
    │   • Pre-trained on WikiText  │
    └──────────────────────────────┘
                   │
                   │
                   ▼
    Step 4: Fine-tune (3 epochs, ~6 hours)
    ┌──────────────────────────────┐
    │  Epoch 1: Loss 0.15          │
    │  Epoch 2: Loss 0.10          │
    │  Epoch 3: Loss 0.08          │
    │  ✓ Best: F1 = 0.544          │
    └──────────────────────────────┘
                   │
                   │
                   ▼
    Step 5: Save Model
    ┌──────────────────────────────────────┐
    │  models/distilbert-goemotions-mental/ │
    │  ├── model.safetensors (255 MB) ⭐    │
    │  ├── config.json                      │
    │  ├── tokenizer.json                   │
    │  ├── vocab.txt                        │
    │  └── ... (other config files)         │
    └──────────────────────────────────────┘
```

```
                    │
                    │ ☑ TRAINING COMPLETE
                    │
┌───────────────────┴──────────────────────────────────────────────────────┐
│                                                                          │
│                      DEPLOYMENT PHASE (App Startup)                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

    Step 1: Start Flask App
    ┌──────────────────────────────┐
    │   python app.py              │
    │   (backend/app.py)           │
    └──────────────────────────────┘
                    │
                    │
                    ▼
    Step 2: Import Predictor
    ┌──────────────────────────────┐
    │  from emotion_predictor      │
    │    import emotion_predictor  │
    └──────────────────────────────┘
                    │
                    │
                    ▼
    Step 3: Load Model (Once)
    ┌──────────────────────────────────────────┐
    │  emotion_predictor.load_model(           │
    │    "./models/distilbert-goemotions..."   │
    │  )                                       │
    │  ✓ Loads model.safetensors into RAM      │
    │  ✓ Loads tokenizer                       │
    │  ✓ ~300 MB memory allocated              │
    └──────────────────────────────────────────┘
                    │
                    │
                    ▼
    Step 4: API Ready
    ┌──────────────────────────────┐
    │  Flask running on :5000      │
    │  Model loaded and waiting    │
    │  Ready to accept requests    │
    └──────────────────────────────┘
                    │
                    │ ☑ APP READY
                    │
┌───────────────────┴──────────────────────────────────────────────────────┐
│                                                                          │
│                       INFERENCE PHASE (Per Request)                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

    REQUEST: User submits text
    ┌──────────────────────────────────────────┐
    │  POST /api/analyze                       │
    │  {                                       │
    │    "text": "I'm anxious about tomorrow"     │
```

```
│  }                                                  │
└──────────────────────────────────────────────────────┘
                  │
                  │
                  ▼
Step 1: Tokenization (emotion_predictor.py)
┌──────────────────────────────────────────────────────┐
│  Text → Tokens                                       │
│  "I'm anxious" →                                     │
│  ["[CLS]", "i", "'", "m", "an", "##xious",  │
│   "[SEP]"]                                           │
│                                                      │
│  Token IDs:                                          │
│  [101, 1045, 1005, 1049, 2019, 22046, 102]  │
└──────────────────────────────────────────────────────┘
                  │
                  │
                  ▼
Step 2: Model Forward Pass (~300ms)
┌──────────────────────────────────────────────────────┐
│     DistilBERT Model                                 │
│                                                      │
│  Input IDs [512 tokens]                              │
│       ↓                                              │
│  Embedding Layer (768-dim)                           │
│       ↓                                              │
│  Transformer Layer 1                                 │
│       ↓                                              │
│  Transformer Layer 2                                 │
│       ↓                                              │
│  ...  (6 layers total)                               │
│       ↓                                              │
│  Pooled Output (768-dim)                             │
│       ↓                                              │
│  Classification Head (768 → 28)                      │
│       ↓                                              │
│  Logits [28 raw scores]                              │
│  [-1.2, 0.5, 2.1, 1.5, ...]                         │
└──────────────────────────────────────────────────────┘
                  │
                  │
                  ▼
Step 3: Sigmoid Activation
┌──────────────────────────────────────────────────────┐
│  Logits → Probabilities (0 to 1)                     │
│                                                      │
│  sigmoid(2.1) = 0.89  (fear)                        │
│  sigmoid(1.5) = 0.82  (nervousness)                 │
│  sigmoid(0.5) = 0.62  (anxiety)                     │
│  sigmoid(-1.2) = 0.23 (joy)                         │
└──────────────────────────────────────────────────────┘
                  │
                  │
                  ▼
Step 4: Threshold Filtering (>0.5)
┌──────────────────────────────────────────────────────┐
│  Keep only emotions with prob ≥ 0.5:                 │
│                                                      │
```

```
│  {                                              │
│    "fear": 0.89,        ✓ (above 0.5)          │
│    "nervousness": 0.82, ✓ (above 0.5)          │
│    "anxiety": 0.62,     ✓ (above 0.5)          │
│    "joy": 0.23          ✗ (below 0.5)          │
│  }                                              │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
Step 5: Mental State Classification
┌─────────────────────────────────────────────────┐
│  Map emotions to mental states:                 │
│                                                 │
│  Depression: max(sadness, grief) = 0.12         │
│  Anxiety: max(fear, nervousness) = 0.89 ⭐      │
│  Stress: max(anger, annoyance) = 0.34           │
│                                                 │
│  → Dominant: "anxiety" (score: 0.89)           │
│  → Severity: "severe" (>0.60)                  │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
RESPONSE: Return JSON
┌─────────────────────────────────────────────────┐
│  {                                              │
│    "emotions": {                                │
│      "fear": 0.89,                              │
│      "nervousness": 0.82,                       │
│      "anxiety": 0.62                            │
│    },                                           │
│    "mental_state": "anxiety",                   │
│    "severity": "severe",                        │
│    "severity_score": 0.89,                      │
│    "state_scores": {                            │
│      "depression": 0.12,                        │
│      "anxiety": 0.89,                           │
│      "stress": 0.34                             │
│    }                                            │
│  }                                              │
└─────────────────────────────────────────────────┘

☑ REQUEST COMPLETE (Total: ~400ms)


┌──────────────────────────────────────────────────────────────────────────┐
│                              KEY FILE MAPPING                            │
└──────────────────────────────────────────────────────────────────────────┘


🗀 Training Files (WHERE MODEL IS CREATED):
   ├── train_model.py              ⭐ Main training script
   ├── train_model_advanced.py     🔬 Advanced training options
   ├── train_model_quick.py        ⚡ Quick test training
   └── evaluate_model.py           📊 Model evaluation
```

```
🗀 Model Files (TRAINED MODEL ARTIFACTS):
   └── models/distilbert-goemotions-mental/
       ├── model.safetensors       ★ 255 MB - Neural network weights
       ├── config.json             ⚙ Model architecture config
       ├── tokenizer.json          📝 Tokenization rules
       ├── vocab.txt               📚 30,522 word vocabulary
       └── ... (other config files)

🗀 Inference Files (USED IN PRODUCTION):
   ├── emotion_predictor.py        ★ Main inference service
   │   └── EmotionPredictor class  (predict_emotions, classify_mental_state)
   │
   ├── app.py                      🌐 Flask API server
   │   └── Loads emotion_predictor at startup
   │
   └── routes/predict.py           🔌 API endpoints
       └── Uses emotion_predictor for predictions


┌──────────────────────────────────────────────────────────────────────────┐
│                            KEY DIFFERENCES                               │
└──────────────────────────────────────────────────────────────────────────┘


╔═══════════════════╦═══════════════════╦═══════════════════════════════════╗
║═══════════════════╤═══════════════════╤═══════════════════════════════════╕
│     Aspect        │   Training        │        Production                 │
╞═══════════════════╪═══════════════════╪═══════════════════════════════════╡
│ File              │ train_model.py    │ emotion_predictor.py              │
│ Purpose           │ Create model      │ Use trained model                 │
│ Frequency         │ Once (or rare)    │ Every request                     │
│ Duration          │ ~6 hours          │ ~400ms per request                │
│ Dataset           │ 58K samples       │ Single user text                  │
│ Output            │ model.safetensors │ Emotion predictions (JSON)        │
│ GPU/CPU           │ GPU preferred     │ CPU works fine                    │
│ Memory            │ ~2-4 GB           │ ~300 MB                           │
│ Computation       │ Training (grads)  │ Inference (no grads)              │
╘═══════════════════╧═══════════════════╧═══════════════════════════════════╛
╚═══════════════════╩═══════════════════╩═══════════════════════════════════╝


┌──────────────────────────────────────────────────────────────────────────┐
│                      ANALOGY: Restaurant Kitchen                         │
└──────────────────────────────────────────────────────────────────────────┘

Training (train_model.py):
    = Recipe Development Phase
    - Chef experiments with ingredients (GoEmotions dataset)
    - Tests different cooking times/temperatures (hyperparameters)
    - Creates the perfect recipe (trained model)
    - Writes it down (saves to model.safetensors)
    - Takes hours/days to perfect
    - Done once, rarely repeated

Production (emotion_predictor.py):
    = Cooking in Restaurant
    - Chef follows the proven recipe (loads trained model)
```

```
    - Customer orders food (user submits text)
    - Chef cooks using recipe (model inference)
    - Serves the dish (returns predictions)
    - Takes minutes per order
    - Repeated many times per day


┌──────────────────────────────────────────────────────────────────────────┐
│                              QUICK ANSWERS                               │
└──────────────────────────────────────────────────────────────────────────┘

❓ What algorithm is used?
   ☑ DistilBERT (66M parameter transformer model)

❓ Which file trains the model?
   ☑ backend/train_model.py (main training script)

❓ Which file is the trained model?
   ☑ backend/models/distilbert-goemotions-mental/model.safetensors
       (255 MB file containing all learned weights)

❓ Which file uses the model in production?
   ☑ backend/emotion_predictor.py (inference service)

❓ Where is the model loaded?
   ☑ app.py imports emotion_predictor, which loads the model
       at startup using load_model()

❓ How long does training take?
   ☑ ~6 hours on CPU, ~1 hour on GPU

❓ How long does prediction take?
   ☑ 300-500ms per text (in production)

❓ Can I retrain the model?
   ☑ Yes! Run: python train_model.py
       (but current model is already trained and working)

❓ Do I need to retrain to use the app?
   ☑ No! The trained model (model.safetensors) is already there
       and ready to use. Just run: python app.py


┌──────────────────────────────────────────────────────────────────────────┐
│                           ARCHITECTURE SUMMARY                           │
└──────────────────────────────────────────────────────────────────────────┘

            DistilBERT Architecture (66M Parameters)

        Input: "I'm feeling anxious" (text string)
                         ↓
        ┌──────────────────────────────────────────┐
        │     Tokenizer (WordPiece)                │
        │  Text → Token IDs [512 max length]       │
```

```
          └──────────────────────────────────────┘
                        │
                        ↓
          ┌──────────────────────────────────────┐
          │   Embedding Layer                    │
          │   Token IDs → Dense vectors (768-dim) │
          └──────────────────────────────────────┘
                        │
                        ↓
          ┌──────────────────────────────────────┐
          │   Transformer Encoder (6 layers)     │
          │   ┌──────────────────────────────┐   │
          │   │ Layer 1: Self-Attention + FFN │   │
          │   │ Layer 2: Self-Attention + FFN │   │
          │   │ Layer 3: Self-Attention + FFN │   │
          │   │ Layer 4: Self-Attention + FFN │   │
          │   │ Layer 5: Self-Attention + FFN │   │
          │   │ Layer 6: Self-Attention + FFN │   │
          │   └──────────────────────────────┘   │
          │   Each layer:                        │
          │   • 12 attention heads               │
          │   • 768 hidden dimensions            │
          │   • 3072 FFN intermediate size       │
          │   • Dropout: 0.1                     │
          └──────────────────────────────────────┘
                        │
                        ↓
          ┌──────────────────────────────────────┐
          │   Pooling (Take [CLS] token output)  │
          │   Sequence → Single vector (768-dim) │
          └──────────────────────────────────────┘
                        │
                        ↓
          ┌──────────────────────────────────────┐
          │   Classification Head                │
          │   Linear: 768 → 28 (one per emotion) │
          └──────────────────────────────────────┘
                        │
                        ↓
          ┌──────────────────────────────────────┐
          │   Sigmoid Activation                 │
          │   Logits → Probabilities (0 to 1)    │
          └──────────────────────────────────────┘
                        │
                        ↓
        Output: [0.12, 0.89, 0.76, ...] (28 probs)
                fear=0.89, nervousness=0.76, ...
```

📝 Summary Table

Component	Training	Production
Script	`train_model.py`	`emotion_predictor.py`
Purpose	Create model	Use model
Input	58K labeled texts	Single user text

Component	Training	Production
Output	`model.safetensors`	Emotion predictions
Duration	~6 hours	~400ms
Frequency	Once	Every request
Model File	Creates it	Loads it

🎯 **Bottom Line:**

- **Training**: `train_model.py` creates `model.safetensors` (255 MB)
- **Production**: `emotion_predictor.py` loads and uses `model.safetensors`
- **Algorithm**: DistilBERT (66M params, 6 layers, 768-dim)
- **Task**: Multi-label emotion classification (28 emotions)